Projeto Integrador II — Cibersegurança e Criptografia

Atividade autônoma — Plano do terminal em C

Prof. Dr. Felix Carvalho Rodrigues — Escola Politécnica, PUC-Campinas

Objetivo. Este documento detalha a atividade autônoma do semestre: a construção, em grupo, de um **terminal simplificado em linguagem C**, versionado em **git/GitHub**, que integra os conteúdos do módulo. O projeto tem uma **entrega parcial** (A_1) e uma **entrega final com defesa** diante de uma banca (A_2).

O que é o terminal

O terminal é um **programa de linha de comando** escrito em C. Ele exibe um **prompt**, lê o que o usuário digita, interpreta a linha como um **comando** e executa a ação correspondente. O ciclo se repete até o usuário sair. Esse tipo de programa é chamado de interpretador de comandos, e é a forma como o próprio sistema operacional recebe ordens do usuário.

O terminal do projeto começa simples e cresce ao longo do semestre. Na primeira etapa ele oferece um pequeno conjunto de comandos de uso geral e uma **cifra de César**. Na etapa final, o grupo acrescenta o **controle de usuários** e uma **segunda cifra**, escolhida pelo grupo, com o mesmo formato de uso.

Foco do semestre. O terminal trabalha com **algoritmos e programação básicos** e com **cifras históricas**. Todas as cifras são implementadas pelo próprio grupo, em C, **sem bibliotecas de criptografia**. O objetivo é entender o funcionamento das cifras, não usar soluções prontas.

Os conteúdos trabalhados

Conteúdo	Como aparece no terminal
Estrutura de programas	Leitura e separação da linha de comando em argumentos, laço de comandos e organização do código em funções
Manipulação de textos e arquivos	Comandos que criam e leem arquivos e o tratamento das letras nos comandos de cifra
Controle de acesso e proteção de dados	Contas de usuário, autenticação por senha, sessão e a forma de guardar dados sensíveis
Cifras clássicas e aritmética modular	Cifra de César, o deslocamento circular no alfabeto e, na etapa final, a cifra escolhida pelo grupo
Padronização, documentação e versionamento	Interface padronizada dos comandos, `README`, histórico de commits e boas práticas de senha

Requisitos técnicos

- Linguagem **C**, compilável com um compilador comum (gcc ou equivalente);
- Código versionado com **git**, com o repositório hospedado no **GitHub** (preferencialmente público), contendo um arquivo `README` com instruções de compilação e uso;
- Cifras **implementadas pelo grupo**; não é permitido usar bibliotecas de criptografia;
- Grupos de **5 a 6 integrantes**, podendo ser os **mesmos integrantes do grupo da atividade de extensão**;
- O projeto é em grupo, mas as **notas são individuais**; cada integrante deve saber explicar qualquer parte do terminal.

Catálogo de comandos

Comando	O que faz	Entrega
`ajuda`	Lista os comandos e mostra a ajuda de um comando	A_1
`sair`	Encerra a sessão e o programa	A_1
`ec`	Repete um texto na tela	A_1
`criar`	Cria um arquivo com um texto	A_1
`cat`	Mostra o conteúdo de um arquivo	A_1
`cesar cifrar`	Cifra um texto com a cifra de César	A_1
`cesar decifrar`	Decifra um texto com a cifra de César	A_1
`registrar`	Cria uma conta de usuário	A_2
`login`	Inicia uma sessão	A_2
`logout`	Encerra a sessão atual	A_2
`quem-sou`	Informa o usuário da sessão	A_2
`<cifra> cifrar`	Cifra um texto com a cifra escolhida pelo grupo	A_2
`<cifra> decifrar`	Decifra um texto com a cifra escolhida pelo grupo	A_2

Especificação dos comandos

Para cada comando são descritos a **sintaxe**, os **argumentos**, o **comportamento esperado**, a **saída**, o **tratamento de erros** e um **exemplo**. O texto entre <> indica um valor a ser informado pelo usuário.

Núcleo do terminal

ajuda

Sintaxe. `ajuda` ou `ajuda <comando>`. **Argumentos.** Opcional: o nome de um comando. **Comportamento.** Sem argumento, lista todos os comandos disponíveis, com uma linha de descrição para cada um. Com um nome de comando, mostra a sintaxe e um exemplo desse comando. **Saída.** Lista ou explicação na tela. **Erros.** Comando inexistente: informa `comando desconhecido`. **Exemplo.** `ajuda cesar` mostra como usar `cesar`.

sair

Sintaxe. `sair`. **Argumentos.** Nenhum. **Comportamento.** Encerra a sessão atual, se houver, e termina o programa. **Saída.** Mensagem de despedida. **Erros.** Nenhum. **Exemplo.** `sair` encerra o terminal.

ec

Sintaxe. `ec <texto>`. **Argumentos.** Um texto, que pode conter espaços entre aspas. **Comportamento.** Imprime o texto na tela. Serve para testar os comandos de cifra. **Saída.** O próprio texto, seguido de nova linha. **Erros.** Sem texto: informa que falta o texto. **Exemplo.** `ec ola mundo` imprime `ola mundo`.

criar

Sintaxe. `criar <arquivo> <texto>`. **Argumentos.** O nome do arquivo e o texto a gravar. **Comportamento.** Cria o arquivo na pasta de trabalho do terminal com o texto informado. Se o arquivo já existir, o conteúdo é substituído. O nome não pode conter caminhos com barras. **Saída.** Confirmação de arquivo criado. **Erros.** Nome vazio, texto ausente ou falha de escrita: informa o problema. **Exemplo.** `criar msg.txt def` cria `msg.txt` com o texto `def`.

cat

Sintaxe. `cat <arquivo>`. **Argumentos.** O nome do arquivo. **Comportamento.** Exibe na tela o conteúdo do arquivo indicado. **Saída.** O conteúdo do arquivo. **Erros.** Arquivo inexistente: informa `arquivo nao encontrado`. **Exemplo.** `cat msg.txt` mostra `def`.

Cifra de César (entrega A_1)

cesar cifrar

Sintaxe. `cesar cifrar -t <texto> <deslocamento>`
`cesar cifrar -f <arquivo> <deslocamento>`. **Argumentos.** `-t` indica que a entrada é um texto; `-f` indica que a entrada é um arquivo. O `<deslocamento>` é um número de 0 a 25. **Comportamento.** Percorre a entrada letra a letra e desloca cada letra no alfabeto em `<deslocamento>` posições, de forma circular (depois do Z vem o A). Letras maiúsculas continuam maiúsculas e minúsculas continuam minúsculas; espaços, números e sinais de pontuação não são alterados. Com `-t`, o resultado aparece na tela; com `-f`, o resultado é gravado no arquivo `<arquivo>.cifrado`. **Saída.** Texto cifrado, na tela ou em arquivo. **Erros.** Deslocamento fora de 0 a 25; arquivo inexistente; falta de argumentos. **Exemplo.** `cesar cifrar -t ABC 3` mostra `DEF`.

cesar decifrar

Sintaxe. `cesar decifrar -t <texto> <deslocamento>`
`cesar decifrar -f <arquivo> <deslocamento>`. **Argumentos.** Como em `cesar cifrar`. **Comportamento.** Faz o caminho inverso: desloca cada letra em `<deslocamento>` posições no sentido contrário. Com `-f`, o resultado é gravado no arquivo `<arquivo>.decifrado`. **Saída.** Texto decifrado, na tela ou em arquivo. **Erros.** Como em `cesar cifrar`. **Exemplo.** `cesar decifrar -t DEF 3` mostra

`ABC`.

Sessão e usuários (entrega A_2)

registrar

Sintaxe. `registrar <usuario>`. **Argumentos.** O nome de usuário a criar. **Comportamento.** Cria uma conta. Solicita a senha duas vezes e a guarda com uma **transformação simples** definida pelo grupo, evitando gravar a senha em texto puro. Recusa um nome de usuário já existente ou vazio. **Saída.** Confirmação do cadastro. **Erros.** Usuário já existe; nome inválido; senhas diferentes. **Exemplo.** `registrar ana` pede a senha e cria a conta `ana`.

login

Sintaxe. `login <usuario>`. **Argumentos.** O nome de usuário. **Comportamento.** Solicita a senha, aplica a mesma transformação usada no cadastro e compara com a armazenada. Em caso de sucesso, inicia a sessão. **Saída.** Sessão iniciada ou aviso de falha. **Erros.** Usuário inexistente; senha incorreta. **Exemplo.** `login ana` pede a senha e entra na conta.

logout

Sintaxe. `logout`. **Argumentos.** Nenhum. **Comportamento.** Encerra a sessão atual, se existir, e volta ao estado deslogado. **Saída.** Confirmação de saída. **Erros.** Sem sessão ativa: informa que não há usuário logado. **Exemplo.** `logout` encerra a sessão de `ana`.

quem-sou

Sintaxe. `quem-sou`. **Argumentos.** Nenhum. **Comportamento.** Informa o usuário da sessão atual. **Saída.** O nome do usuário ou um aviso de que não há sessão. **Erros.** Nenhum. **Exemplo.** `quem-sou` mostra `ana`.

Segunda cifra, à escolha do grupo (entrega A_2)

O grupo escolhe **uma** cifra e a implementa com o **mesmo formato** usado na cifra de César: um comando `cifrar` e um comando `decifrar`, aceitando a entrada por texto (`-t`) ou por arquivo (`-f`) e uma chave. O grupo deve **documentar a matemática da cifra** e **definir e explicar o formato da chave**. Sugestões:

- **Cifra afim**: a chave é um par de números `a b`, com `a` coprimo de 26. Exemplo: `afim cifrar -t ABC 5 8`;
- **Cifra de Hill**: a chave é uma matriz quadrada (2×2 ou 3×3) com determinante invertível módulo 26. Exemplo: `hill cifrar -t ABC ...`;
- **Tabela de substituição**: a chave é uma permutação das 26 letras do alfabeto. Exemplo: `subst cifrar -t ABC QWERTYUIOPASDFGHJKLZXCVBNM`;
- **Outra cifra**: livre, desde que combinada antes com o professor.

Justificativa obrigatória. O relatório da entrega final deve explicar **por que** o grupo escolheu a cifra, **como** ela funciona (com a matemática envolvida) e **quais os seus limites**, comparando com a cifra de César.

Papéis e organização do grupo

Sugere-se distribuir responsabilidades, por exemplo: estrutura do programa e comandos do núcleo; sessão e usuários; comandos de cifra; documentação e repositório. As tarefas podem girar ao longo do semestre, mas **todos** devem conhecer o código, porque a defesa na banca é individual.

Cronograma preliminar

Período	Atividades	Marco
05/10 a 06/11	Formação dos grupos, criação do repositório, núcleo do terminal e cifra de César	—
09/11	Protótipo no GitHub e relatório	**Entrega** A_1
09/11 a 26/11	Sessão e usuários, segunda cifra, ajustes, testes e documentação final	—
27/11	Terminal completo e relatório final	**Entrega** A_2
A definir	Demonstração e defesa perante a banca	Banca

Obs.: as datas podem ser ajustadas pelo professor ao longo do semestre.

Critérios de avaliação

A nota da atividade autônoma é $A = \dfrac{A_1 + 3 \cdot A_2}{4}$. Os critérios gerais são:

- **Funcionamento**: os comandos fazem o que a especificação pede, inclusive nos casos de erro;
- **Cifras**: a cifra de César e a cifra escolhida estão corretas e são implementadas pelo grupo;
- **Uso de git**: repositório com histórico de commits, `README` e evolução visível do código;
- **Relatório**: descreve o funcionamento, os conceitos aplicados e o estado do projeto com clareza;
- **Demonstração e defesa**: o terminal é demonstrado e cada integrante responde sobre qualquer parte do projeto.